

计算机程序设计 · 2026 年真题（回忆版 + 同题型复盘详解）

类型：考生回忆版 + 同题型复盘 | 满分 200 分 · 150 分钟

科目：广东普通专升本《计算机基础与程序设计》

参考教材：谭浩强《C语言程序设计》（第5版）+ 严蔚敏《数据结构（C语言版）》（第2版）

试卷结构（满分200分，150分钟）

题型	题量	分值	核心考查内容
一、单项选择题	20题	60分（3分/题）	C语言语法、表达式、程序阅读、数据结构概念
二、判断题	10题	20分（2分/题）	C语言细节、数据结构性质、算法复杂度
三、填空题	5题	20分（4分/题）	程序补全、字符串、树与图
四、简答题	4题	40分（10分/题）	概念辨析、存储结构、算法分析
五、计算题	3题	30分（10分/题）	程序执行结果、查找与排序过程
六、应用题	3题	30分（10分/题）	C语言程序设计与数据处理

本页含两类内容，别混：

- 2026 真题原题 —— 3 道应用题，有视频源逐帧/逐句核对，见第一节；

- 同题型复盘题 —— 16 道按 2026 考纲题型自编的练习，**不是真题**，见第二、三节。

一、2026 真题 · 考生回忆版（应用题 3 题）

来源：B 站 [BV1Z4w3znE6h](#) P5（UP：一灯computer）· 考生回忆版

核实时间：2026-09-19，逐帧 + 逐句核对

边界：UP 在片尾自述「目前来说我看到比较完善的就是这三道编程题」—— 2026 计算机公开可考的回忆题**只有这 3 道**，其余题型暂无真题源。

⚠ 数据说明：第 1、2 题视频里给的那组具体数值，是**讲解老师自己编的**（原话：「我不确定他有没有给到我这个.....所以呢我就自己随机写了一个分数过来」「他也没有给我值啊，没有给我具体的值，我就假设.....」）。真题原卷只给了题目要求，没给数据。所以下面把「题目要求」和「讲解者自编的示例数据」**分开写**—— 别把示例数据当成题目条件背下来。

第 1 题 • 射箭比赛计分（数组 + 循环 + 分支）

题目要求

有 A、B 两个小队，每队 5 名队员进行射箭比赛，成绩用数组存储。按队员顺序逐个比较两队的分数：A 队队员分数高于 B 队则记 A 队赢一局，低于则记 B 队赢一局，相等记平局。要求统计并输出 A 队赢的局数、B 队赢的局数、平局的次数。

⚠ 本题没有卷面帧 —— 上面这段是按口播整理的，不是卷面逐字原文。

示例数据（讲解者自编，非真题数据）

队伍	队员 1	队员 2	队员 3	队员 4	队员 5
A 队	15	7	20	22	21
B 队	16	5	20	22	14

参考代码

```
#include <stdio.h>

int main() {
    int a[5] = {15, 7, 20, 22, 21};
```

```
int b[5] = {16, 5, 20, 22, 14};
int a_win = 0, b_win = 0, draw = 0;
int i;

for (i = 0; i < 5; i++) {
    if (a[i] > b[i])        a_win++;    /* A 队这一局赢 */
    else if (a[i] < b[i])   b_win++;    /* B 队这一局赢 */
    else                    draw++;     /* 平局单独统计 */
}

printf("A队赢 %d 局, B队赢 %d 局, 平局 %d 次\n", a_win, b_win, draw);
return 0;
}
```

运行结果（在上面那组自编数据下）

A队赢 2 局, B队赢 1 局, 平局 2 次

易错点

- 三个计数器写成同一个变量 —— 三个统计量必须各自独立。
- 判断写成 `a[i] >= b[i]` 归到「赢」 —— **相等必须单独归入平局**，否则平局数恒为 0。
- 循环边界写成 `i <= 5` —— 数组长度 5，合法下标 0~4，会越界。

第 2 题 • 停车场七天最多台数（指针）

题目要求（卷面逐字）

停车场连续七天记录每晚停车的台数，要求找出一周内停车的最多是多少台，用指针实现。

示例数据（讲解者自编，非真题数据）

28, 17, 19, 43, 22, 85, 8

参考代码

```
#include <stdio.h>

int main() {
    int car[7] = {28, 17, 19, 43, 22, 85, 8};
    int *p = car;          /* 数组名就是首元素地址 */
    int max = *p;          /* 先假设第一天最多，而不是写 max = 0 */
    int i;

    for (i = 1; i < 7; i++) {
        p++;                /* 指针后移一个元素 */
        if (*p > max) max = *p;
    }

    printf("一周内停车最多 %d 台\n", max);
    return 0;
}
```

更地道的写法是把指针移动直接放进 for :

```
for (p = car + 1; p < car + 7; p++)
    if (*p > max) max = *p;
```

运行结果

一周内停车最多 85 台

易错点

- `max` 初值写 `0` —— 本题数据全是正数才碰巧对，一旦数据可能为负就错。规范做法是拿第一个元素当初值。
- 指针忘了初始化（`int *p = car;` 或 `&car[0]`）就直接 `*p` —— 野指针。
- `*p++` 与 `(*p)++` 混淆：前者是「取值后指针后移」，后者是「取到的那个值加一」。
- 循环边界写成 `i <= 7` —— 越界（合法下标 0~6）。

第 3 题 • 结构体 `staff` 录入与输出

题目要求（卷面逐字）

定义一个结构体 `staff`，里面记录了员工的姓名、id、入职年份和薪资。然后输入一条数据，并且打印出该信息，用结构体编程实现。

参考代码

```
#include <stdio.h>

struct staff {
    char  name[20];
    char  id[20];      /* ★ 用字符数组，不用 int —— 见易错点 */
    int   year;
    float salary;
};

int main() {
    struct staff s;

    printf("请输入 姓名 id 入职年份 薪资：");
    scanf("%s %s %d %f", s.name, s.id, &s.year, &s.salary);

    printf("员工信息如下：\n");
    printf("姓名： %s\n", s.name);
```

```
printf("id: %s\n", s.id);
printf("入职年份: %d\n", s.year);
printf("薪资: %.2f\n", s.salary);

return 0;
}
```

运行示例

请输入 姓名 id 入职年份 薪资: 小李 101 2023 8000

员工信息如下:

姓名: 小李

id: 101

入职年份: 2023

薪资: 8000.00

本题**不预设数据**—— 题目要求就是「输入一条数据」，上面只是示例输入，输出随输入变化。

易错点

- ★ `id` 用 `int` 会丢前导零: 输入 `001` 出来变 `1`。讲解老师实测踩过这个坑，最后改成字符数组 `char id[20]` —— 这样还能存带字母的编号。
- `struct` 定义末尾漏分号 —— 编译报错，而且报错位置常常指向下一行，容易看错。
- 声明变量时漏写 `struct`: C 语言里必须写 `struct staff s;`，除非用 `typedef`。
- `scanf` 读 `float` 用 `%f`、读 `double` 用 `%lf` —— 用混会读出垃圾值。
- 字符串成员用 `=` 赋值 (`s.name = "小李";`) —— 数组名不可赋值，必须用 `scanf` 或 `strcpy`。

二、C语言同题型复盘

1. 以下程序段的输出结果是 ()

```
int a = 5, b = 2;
double c = a / b;
printf("%.2f", c);
```

A. 2.00 B. 2.50 C. 2.5 D. 2

答案：A

解析： `a` 和 `b` 都是 `int` 类型，`a / b` 执行整数除法，结果为 2（小数部分被截断）。虽然赋值给 `double` 变量，但截断已经发生在整数除法阶段。所以输出 2.00。

💡 **秒懂技巧：** 计算机里整数除法就像分蛋糕——“5块蛋糕分给2个人，每人只能拿整块”，结果是2块，剩下的1块丢了。想得到精确值，必须先把其中一个数变成小数（如 `(double)a / b`）。

易错点： 很多同学以为赋值给 `double` 会自动保留小数，但C语言是从右向左执行——先算除法（整数），再赋值。

2. 执行 `int x = 5; printf("%d", x++ + ++x);` 后，输出结果是（）

A. 10 B. 11 C. 12 D. 未定义行为

答案：D

解析： 在同一条表达式中，对同一个变量 `x` 同时进行读取和修改（`x++` 读取后修改，`++x` 修改后读取），且修改顺序未定义，属于**未定义行为**。不同编译器结果可能不同。

💡 **秒懂技巧：** 就像你同时说“先吃饭再洗碗”和“先洗碗再吃饭”——这两件事顺序冲突，结果不确定。考试中遇到这种情况，选“未定义”就对了。

铁律： 一条语句中，一个变量最多被修改一次。

3. 以下代码的输出是 ()

```
char s[] = "Hello";
char t[] = "Hello";
if (s == t) printf("相等");
else printf("不相等");
```

- A. 相等 B. 不相等 C. 编译错误 D. 运行时错误

答案：B

解析： `s` 和 `t` 是两个不同的数组，它们在内存中占据不同的地址。`s == t` 比较的是**数组首地址**，而不是字符串内容。比较字符串内容应使用 `strcmp`。

 **秒懂技巧：** 就像两个人都有"Hello"的笔记本，但笔记本是不同本子——问"这两本笔记本是不是同一本"，答案是"不是"。要比较内容，得逐字对照（`strcmp`）。

延伸： `strlen(s)` 计算长度（不含 `\0`），`sizeof(s)` 计算数组大小（含 `\0`），`sizeof("Hello") = 6`。

4. 执行以下代码，输出结果是 ()

```
int a[3][3] = {{1,2},{3,4},{5,6}};
printf("%d", a[1][1]);
```

- A. 1 B. 2 C. 3 D. 4

答案：D

解析：二维数组初始化时，未指定的元素自动补 0。 `a[3][3]` 实际存储为：

第0行：{1, 2, 0}

第1行：{3, 4, 0}

第2行：{5, 6, 0}

`a[1][1]` 是第1行第1列，即 4。

💡 **秒懂技巧：**二维数组就像 Excel 表格。`a[1][1]` 是第2行第2列（C语言从0开始数）。没填的格子自动填0。

5. 以下程序输出是（ ）

```
void swap(int x, int y) {  
    int t = x; x = y; y = t;  
}  
  
int main() {  
    int a = 3, b = 5;  
    swap(a, b);  
    printf("%d,%d", a, b);  
    return 0;  
}
```

A. 3,5 B. 5,3 C. 3,3 D. 5,5

答案：A

解析：C语言函数参数是**值传递**—— `swap(a, b)` 把 `a` 和 `b` 的**值拷贝**给 `x` 和 `y`，函数内部交换的是拷贝值，不影响原变量。

💡 **秒懂技巧：** 就像你把身份证复印件给工作人员，工作人员在复印件上改名字——原件上的名字不会变。想改原件，必须给原件地址（指针）。

要真正交换，必须用指针：

```
void swap(int *x, int *y) {  
    int t = *x; *x = *y; *y = t;  
}  
swap(&a, &b); // 传地址
```

6. 执行下面程序段，输出结果是（ ）

```
int i, sum = 0;  
for (i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    sum += i;  
}  
printf("%d", sum);
```

A. 9 B. 12 C. 15 D. 18

答案：B

解析： `i==3` 时跳过本轮累加，因此 `sum=1+2+4+5=12`。

💡 **秒懂技巧：** `continue` 就像“跳过当前这一轮”——`i=3` 时直接跳过去下一轮。`break` 是“整个循环都不玩了”。

易错点： `continue` 只跳过当前循环，`break` 才会直接结束循环。

7. 设 `int x=10; int *p=&x;` , 执行 `(*p)++;` 后 ()

- A. `p` 的地址加 1 B. `x` 变为 11 C. `p` 指向空地址 D. 语法错误

答案：B

解析：括号保证先执行 `*p` 得到 `x` , 再对该变量自增, 所以 `x` 变为 11。

💡 秒懂技巧：指针就像"门牌号"—— `p` 是写着"x的家"的门牌, `*p` 是去敲这个门。`(*p)++` 是"去敲x的门, 然后让x加1"。`p++` 是"门牌号本身加1", 指向别处了。

区分： `(*p)++` 修改指向的值, `p++` 修改指针本身。

8. 关于编译错误与运行错误, 下列说法正确的是 ()

- A. 数组越界一定能在编译阶段发现
B. 缺少分号通常属于编译错误
C. 除以 0 一定属于链接错误
D. 函数声明不匹配一定不会报警

答案：B

解析：缺少分号会破坏语法, 通常由编译器报告。数组越界和除以 0 可能在运行阶段才暴露; 链接错误常见于函数或全局符号找不到定义。

💡 秒懂技巧：

- 编译错误 = 写作文时语法不通 (老师一眼看出)
- 链接错误 = 你引用了一本不存在的书 (图书馆找不到)

- **运行错误** = 作文写好了，但读的时候发现逻辑矛盾（只有读了才知道）

考点： 2026 考纲新增强调的程序运行环境与代码调试。

三、数据结构同题型复盘

9. 长度为 n 的顺序表，在表头插入一个元素的时间复杂度是（ ）

- A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n^2)$

答案：C

解析： 在表头插入时，原有 n 个元素都需要向后移动一位，因此时间复杂度为 $O(n)$ 。

💡 **秒懂技巧：** 就像排队时插到最前面——后面所有人必须往后退一步。人越多，退的步数越多。

10. 栈的操作原则是（ ）

- A. 先进先出 B. 后进先出 C. 随机存取 D. 按关键字有序

答案：B

解析： 栈是后进先出（LIFO）的受限线性表，只允许在栈顶进行插入和删除。

💡 **秒懂技巧：** 栈就像一摞盘子——你只能从最上面拿盘子（后放的先拿），也只能在最上面放盘子。

11. 循环队列中，若队头指针为 `front`、队尾指针为 `rear`、数组长度为 `m`，常用的队满条件是（ ）

- A. `front == rear`
- B. `(rear + 1) % m == front`
- C. `rear == m`
- D. `front == 0`

答案：B

解析：采用"牺牲一个存储单元"的实现时，`front == rear` 表示队空，`(rear+1)%m == front` 表示队满。

💡 **秒懂技巧：** 循环队列就像一圈停车位——你有一圈车位，队尾指针追上了队头指针就说明满了。但为了区分"空"和"满"，故意空一个车位不占。

12. 一棵具有 7 个结点的完全二叉树，其叶子结点数是（ ）

- A. 2
- B. 3
- C. 4
- D. 7

答案：C

解析：7 个结点恰好构成三层满二叉树，第三层的 4 个结点都是叶子。

💡 **秒懂技巧：** 画个三层二叉树：第一层1个，第二层2个，第三层4个。第三层没有子节点的就是叶子。

13. 对二叉搜索树进行哪一种遍历可以得到递增序列（ ）

- A. 先序遍历
- B. 中序遍历
- C. 后序遍历
- D. 层序遍历

答案：B

解析： 二叉搜索树满足"左子树关键字小于根，右子树关键字大于根"，中序遍历顺序是左—根—右，因此结果递增。

💡 **秒懂技巧：** 中序遍历就像"先看左边 → 再看中间 → 再看右边"，因为左边都是小的，所以从小到大输出。

14. 含 n 个顶点的无向连通图，至少有多少条边（ ）

A. $n-1$ B. n C. $n+1$ D. $2n$

答案：A

解析： 无向连通图的极小连通结构是一棵生成树，树有 $n-1$ 条边。

💡 **秒懂技巧：** 想象你要用网线把 n 台电脑连起来，让它们都能互相通信——最少需要 $n-1$ 根网线（形成一条线或树形结构）。

15. 对长度为 n 的有序表进行折半查找，平均时间复杂度是（ ）

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

答案：B

解析： 每次比较都会排除一半查找区间，因此比较次数与 $\log_2 n$ 同阶。

💡 **秒懂技巧：** 折半查找就像猜数字——你说"50"，我说"大了"，你就知道答案在1-49之间，一次排除一半。100个数字最多猜7次就能找到。

前提： 表必须有序，并且存储结构支持随机访问。

16. 下列排序算法中，平均时间复杂度为 $O(n\log n)$ 且通常需要额外 $O(n)$ 空间的是（ ）

- A. 冒泡排序 B. 直接插入排序 C. 归并排序 D. 简单选择排序

答案：C

解析：归并排序通过分治将序列不断二分，再使用辅助数组归并，平均和最坏时间复杂度均为 $O(n\log n)$ 。

💡 秒懂技巧：归并排序就像“分组比赛”——先把一堆牌分成两小堆，每小堆排好，再合并成有序的一堆。合并时需要一张额外的桌子放牌（额外空间）。

答案速查（第二、三节的同型复盘题）

第一节的 3 道真题原题没有固定答案 —— 输出取决于输入数据。

题号	答案	核心考点
1	A	整数除法与类型转换
2	D	未定义行为
3	B	数组地址比较 vs 字符串比较
4	D	二维数组初始化
5	A	值传递（参数传递方式）
6	B	循环与 <code>continue</code>
7	B	指针解引用
8	B	编译、链接与运行错误
9	C	顺序表插入时间复杂度
10	B	栈的 LIFO 特性
11	B	循环队列队满条件

12	C	完全二叉树叶子结点
13	B	二叉搜索树中序遍历
14	A	无向连通图最少边数
15	B	折半查找时间复杂度
16	C	归并排序时空复杂度

2026 考纲新变化

变化项	说明
改错题信号	2026 题型可能含改错题，2027 备考需关注
程序调试	考纲新增强调程序运行环境与代码调试
参考教材更新	严蔚敏、吴伟民《数据结构（C 语言版）》第 2 版（ 清华大学出版社 2025-10 ，  官方考纲确认；原"人邮 2021"为旧口径）